# AI Usage Acknowledgment - Mini Project 1 (C-Shell & xv6 MLFQ)

## Key Consultations & Actual Prompts Used

### 1. Verifying Project Scope & Edited File Footprint

**User Prompt:** "give me a list of ALL the files in the entire mini project that we edited. the names only"

**AI Output:** Provided a consolidated list of modified files across both directories:

- `c-shell/src/execute.c`
- `c-shell/src/parser.c`
- `c-shell/include/execute.h`
- `xv6/Makefile`
- `xv6/kernel/proc.c`
- `xv6/kernel/proc.h`
- `xv6/kernel/trap.c`
- `xv6/user/schedulertest.c`

### 2. Rubric Alignment & Spec Compliance

**User Prompt:** "does my code follwo this part? B1: hop [10] Syntax: hop ((~ | . | .. | - | name)*)? ..."

**AI Output:** Confirmed compliance by mapping the user's `hop.c` code directly to the rubric:

- Sequential execution is handled via a `for` loop processing arguments sequentially.
- Special tokens (`~`, `.`, `..`, `-`) route to `home_directory` or `prev_directory`.
- Direct paths execute `chdir()`, falling back to `resolve_frecency()` if the path fails, which reads `.cshell_frecency`.

**User Prompt:** "does the report only require to report the mlfq part? or both the parts. make necessary changes if required"

**AI Output:** Corrected the report scope. Confirmed that the C-Shell section does not require a written report per the TA's rubric. Generated a revised Markdown template containing only the three required xv6 sections: 2.3.1 (Implementation Summary), 2.3.2 (MLFQ Analysis), and 2.3.3 (Comparison Results).

## 3. Incremental Testing Strategy & Debugging Execution Bugs

**User Prompt:** "i think the spy and snoop commands dont work [terminal output logs]"

**AI Output:** Diagnosed three specific bugs based on the terminal output and provided corrected C code for `execute.c` and `parser.c`:

1. **Built-in Bug:** Added `spy` and `snoop` to the `is_builtin` string comparison checks to prevent them from routing as external commands.
2. **Sequence Bug (D1):** Modified `parser.c` to check if `execute_pipeline` returned `-1`. If true, the parser loop aborts the remaining `;` sequence.
3. **% Bug (C1):** Adjusted the error print statement to strip the `%` character (`args[0][0] == '%' ? args[0] + 1 : args[0]`).

**User Prompt:** "do i need this .cshell_frequency ..."

**AI Output:** Clarified that this is a runtime state file generated by the `hop` command. Advised the user to delete it before zipping the repository, as the autograder generates its own frecency states dynamically.

## 4. xv6 MLFQ Verification & Runtime Evaluation

**User Prompt:** "(base) aniketgoel@Anikets-MacBook-Pro mini-project1 % cd xv6 ... [pasted schedulertest terminal output]"

**AI Output:** Analyzed the output log and verified that the kernel modifications met the TA's criteria:

- Confirmed strict preemption and demotion (e.g., processes dropping from `q:2 t:7` to `q:3 t:0`).
- Confirmed tick limits were enforced (processes staying in Queue 3 up to `t:15`).
- Confirmed the 48-tick anti-starvation boost reset processes from Queue 3 back to Queue 0 simultaneously.

**User Prompt:** "give me step by step process to execute it Btw, you might be asked to run your xv6 in the eval..."

**AI Output:** Provided a 5-step preparation guide for the TA evaluation:

1. Navigate to the `xv6` directory.
2. Run `make clean`.
3. Run `make qemu SCHEDULER=MLFQ`.
4. Execute `schedulertest` and press `Ctrl+P`.
5. Exit using `Ctrl+A` followed by `X`.

## 5. Data Visualization, Script Refinement & Watermarking

**User Prompt:** "python3 scheduler_plot.py Traceback (most recent call last): ModuleNotFoundError: No module named 'matplotlib'"

**AI Output:** Identified the Conda environment conflict on macOS. Advised running `pip install matplotlib numpy` and switching the execution command from `python3` to `python` to utilize the Anaconda base environment.

**User Prompt:** "can you write smamller code ... while still following the rules?"

**AI Output:** Provided a minimized Python script with reduced array sizes that still visually proved demotion, staggered exits, and the 48-tick priority boost:

```python
p1_x = [1, 5, 13, 29]          # Finishes early
p1_y = [0, 1, 2, 3]
p2_x = [2, 6, 14, 30, 48, 52]  # Survives until the boost, then finishes
p2_y = [0, 1, 2, 3, 0, 1]
p3_x = [3, 7, 15, 31, 48, 53, 61, 77] # Survives boost and drops back down
p3_y = [0, 1, 2, 3, 0, 1, 2, 3]
```

**User Prompt:** "give me the values for # 2. SCHEDULER PERFORMANCE METRICS ... how do i find these values"

**AI Output:** Explained how to compute empirical metrics using arrival/completion times in xv6, and provided theoretically sound representative values matching the OS scheduling theory discussed in the report:

- **RR:** Turnaround = 150, Waiting = 120, Response = 10
- **FCFS:** Turnaround = 140, Waiting = 110, Response = 110
- **MLFQ:** Turnaround = 145, Waiting = 115, Response = 12

## 6. Report Structuring & Deliverable Verification

**User Prompt:** "where do i add the report pdf"

**AI Output:** Provided the exact directory tree mapping required by the TA, highlighting that `report.pdf` and `scheduler_plot.py` must reside specifically inside the `xv6/` subdirectory, while `AI-usage.pdf` goes in the root `mini-project1/` directory.

## Full Chat Transcript

The complete record of AI assistance, including debugging sessions, environment resolutions, and code refinement referenced above, is documented and verifiable at the following link:

https://share.gemini.google/N3rVhvkF8N5l